

[illegible]

[Stats] 4. Worked Paging Example

- **Problem:** A 32-bit virtual address system has a page size of 4 KB. Main memory is 512 MB, and the system uses a single-level page table. Each page table entry contains the frame address plus 4 auxiliary bits.
- **Goal:** Find the page table size.

Step 1: Extract Bit Lengths

- Virtual Address Length $V = 32$ \text{ bits} $\implies \text{VAS} = 2^{32}$ \text{ bytes}.
- Page Size $S = 4$ \text{ KB} $= 2^{12}$ \text{ bytes} $\implies$ Offset offset $d = 12$ \text{ bits}.
- Physical Memory Size $M = 512$ \text{ MB} $= 2^{29}$ \text{ bytes} $\implies P = 29$ \text{ bits}.

Step 2: Calculate Page & Frame Bits

- Page Number bits: $p = V - d = 32 - 12 = 20 \text{ bits}$ $\implies N_{\text{pages}} = 2^{20}$
- Frame Number bits: $f = P - d = 29 - 12 = 17 \text{ bits}$

Step 3: Calculate PTE Size

$\text{PTE Size} = f + 4 \text{ aux bits} = 17 + 4 = 21 \text{ bits}$ * Since page tables must be byte-addressable, we round 21 bits up to the nearest byte boundary:
 $\text{PTE Size rounded} = 3 \text{ bytes (24 bits)}$

Step 4: Calculate Page Table Size (PTS)

$$\text{PTESize} = \frac{\text{Pages} \times \text{PTESize}}{1024} = \frac{2^{20} \times 3}{1024} = 3 \text{ MB}$$

OS Common Traps

- **PTE Rounding:** Always check if a question asks for the exact bit-level size of the page table or requires byte rounding. In real hardware, PTEs are rounded to whole bytes (e.g., 2, 3, or 4 bytes).
- **Multi-Level Page Table Levels:** If page size is 2^d bytes and page table entry is e bytes, a single page can hold $\frac{2^d}{e}$ entries. In multi-level paging, the size of each level's table is bounded to fit exactly inside a single page frame!
- **Belady's Anomaly:** Remember that increasing the number of page frames can *increase* the page fault rate under FIFO, but **never** under LRU or Optimal replacement!